

SERVICIO NACIONAL DE APRENDIZAJE — SENA
Centro de Servicios y Gestión Empresarial — Regional Antioquia

MANUAL TÉCNICO

Óleo & Lienzo — Galería de venta de pinturas

Proyecto integrador React + Vite · FastAPI · PostgreSQL

Programa de formación	Tecnólogo en Análisis y Desarrollo de Software (código 228118)
Ficha	3406211
Trimestre / Ambiente	03 / 702
Aprendiz	Jerson Jesús Vega Naranjo
Instructores	César Augusto Moreno Mena · Jhan Hader Muñoz
Repositorio	https://github.com/vegajerson88-droid/Oleo-Lienzo
Fecha de entrega	29 de septiembre de 2026
Versión del documento	1.0

Medellín, Antioquia · Colombia

Tabla de contenido

1. Introducción y descripción general del proyecto 4

1.1 El problema que resuelve 4

1.2 Contexto de la formación 4

1.3 Usuarios y actores del sistema 4

2. Objetivos 5

2.1 Objetivo general 5

2.2 Objetivos específicos 5

3. Alcance del proyecto 6

3.1 Funcionalidades incluidas 6

3.2 Funcionalidades excluidas explícitamente 6

4. Arquitectura de la solución 7

4.1 Visión general 7

4.2 Stack tecnológico 7

4.3 Organización del código 8

5. Modelo de datos 9

5.1 Diagrama entidad-relación 9

5.2 Decisiones de modelado que conviene explicar 9

5.3 Diccionario de datos 9

6. Diseño de la solución 16

6.1 Historias de usuario y su implementación 16

6.2 El flujo comercial, paso a paso 16

6.3 Máquinas de estado 16

7. Manual de instalación y configuración 18

7.1 Requisitos previos 18

7.2 Paso 1 — Obtener el código 18

7.3 Paso 2 — Crear la base de datos 18

7.4 Paso 3 — Configurar el backend 18

7.5 Variables de entorno del backend	18
7.6 Paso 4 — Poblar la base y arrancar el backend	19
7.7 Paso 5 — Configurar y arrancar el frontend	19
7.8 Comprobación de que todo quedó bien	19
7.9 Credenciales de prueba	19
8. Documentación técnica de la API	20
8.1 Cómo autenticarse	20
8.2 Ejemplo completo: registrar una venta	20
8.3 Formato uniforme de los errores	20
8.4 Catálogo completo de operaciones	21
9. Manual de usuario	24
9.1 Zona pública	24
9.2 Registro e inicio de sesión	26
9.3 Panel del administrador	27
9.4 Panel del empleado	34
9.5 Panel del cliente	35
9.6 Chatbot de atención	37
9.7 Documentación interactiva de la API	37
10. Pruebas realizadas	39
10.1 Estrategia	39
10.2 Qué cubre cada archivo	39
10.3 Casos de prueba destacados	39
10.4 Cómo ejecutarlas y qué resultado dan	39
10.5 Pruebas manuales complementarias	40
11. Conclusiones y recomendaciones	41
11.1 Aprendizajes del proceso	41
11.2 Limitaciones conocidas	41
11.3 Recomendaciones para una versión futura	41
11.4 Conclusión	41
12. Anexos	43

12.1 Repositorio y estructura de entrega 43

12.2 Documentación complementaria en el repositorio 43

12.3 Scripts de apoyo 43

12.4 Glosario 43

1. Introducción y descripción general del proyecto

1.1 El problema que resuelve

Las galerías de arte pequeñas y medianas de Colombia gestionan su operación con herramientas dispersas: el catálogo en redes sociales, las ventas en un cuaderno o una hoja de cálculo, las facturas en documentos sueltos y la atención al cliente por mensajería. Esa dispersión provoca tres problemas concretos: no hay una fuente única sobre qué obra sigue disponible, no se puede saber cuánto se vendió en un periodo sin reconstruirlo a mano, y el cliente no tiene forma de consultar el estado de su compra sin escribir y esperar respuesta.

Óleo & Lienzo es una aplicación web que resuelve esas tres carencias sobre una única base de datos: publica el catálogo, registra las ventas con su desglose económico, emite facturas con consecutivo, genera reportes en PDF y Excel, muestra indicadores por rol y atiende consultas mediante un chatbot con inteligencia artificial.

1.2 Contexto de la formación

El proyecto se desarrolló durante el tercer trimestre de la ficha 3406211 del programa de Análisis y Desarrollo de Software. Es la evolución acumulada de cinco entregas: la interfaz en React, los estilos con Tailwind, la integración con un backend y base de datos, la migración de ese backend a FastAPI y, por último, la capa comercial, la analítica y la inteligencia artificial.

1.3 Usuarios y actores del sistema

Actor	Quién es	Qué hace en el sistema
Visitante	Cualquiera que entra al sitio sin registrarse.	Recorre la galería y el catálogo, consulta la ficha de cada obra, escribe por el formulario de contacto y conversa con el chatbot.
Cliente	Visitante que creó una cuenta.	Arma pedidos, consulta sus compras y facturas, descarga el PDF de una factura y radica PQR con seguimiento de estado.
Empleado	Personal operativo de la galería.	Gestiona el catálogo, atiende pedidos, registra ventas, emite facturas, genera reportes y responde PQR. No accede a la gestión de usuarios.
Administrador	Responsable de la galería.	Todo lo del empleado, más la gestión de usuarios y roles, el dashboard completo con facturación e ingresos, y el diagnóstico del sistema.

2. Objetivos

2.1 Objetivo general

Desarrollar y desplegar una aplicación web full stack para la gestión comercial de una galería de arte, construida sobre la arquitectura React + Vite → FastAPI → PostgreSQL, que permita administrar el catálogo, registrar ventas, emitir facturas, generar reportes, analizar la información mediante dashboards diferenciados por rol y atender a los clientes con un chatbot basado en inteligencia artificial, aplicando prácticas de seguridad en la autenticación, la autorización y el manejo de credenciales.

2.2 Objetivos específicos

1. Diseñar e implementar una API REST con FastAPI que exponga los recursos del dominio siguiendo los criterios REST de recurso, verbo, ruta y código de respuesta.
2. Modelar la información en una base de datos relacional PostgreSQL con integridad referencial, restricciones de unicidad y reglas de validación en el propio motor.
3. Implementar autenticación con JSON Web Tokens y autorización por roles y permisos, de modo que la decisión final sobre quién puede hacer qué la tome siempre el backend.
4. Construir el módulo comercial completo: pedidos, ventas con desglose de IVA, facturación con consecutivo y generación de reportes en formato PDF y Excel.
5. Desarrollar dashboards diferenciados por rol que obtengan sus indicadores y gráficos de consultas agregadas en la base de datos, sin ningún dato escrito a mano.
6. Integrar inteligencia artificial en dos frentes: un modelo propio de regresión entrenado con el catálogo y un chatbot conectado a un proveedor externo, con las credenciales gestionadas por variables de entorno.
7. Verificar el comportamiento del sistema con una suite de pruebas automatizadas y preparar el despliegue en un entorno de producción en la nube.

3. Alcance del proyecto

3.1 Funcionalidades incluidas

Módulo	Qué incluye
Catálogo	CRUD completo de obras y servicios, con búsqueda por texto y filtros por artista, técnica, disponibilidad y rango de precio, más paginación.
Usuarios y seguridad	Registro, inicio de sesión con JWT, recuperación de contraseña, hashing con bcrypt, tres roles con permisos granulares y cambio de estado activo/inactivo.
Comercial	Pedidos del cliente, confirmación que genera la venta, ventas con desglose de IVA al 19 %, control de inventario y máquina de estados validada.
Facturación	Emisión con consecutivo OL-NNNNNN, congelado de los importes y de los datos del cliente, consulta por varios criterios y descarga en PDF.
Reportes	Reporte diario de ventas en JSON, PDF y Excel, este último con fórmulas y autofiltro para seguir analizando los datos.
Analítica	Dashboards por rol con tarjetas de indicadores, gráfico lineal y de barras, y filtros por fecha, obra, servicio, estado y cliente.
PQR	Radicación con número de radicado, cuatro estados de seguimiento y respuesta por parte del personal.
Inteligencia artificial	Chatbot con Groq que responde usando el catálogo real, y modelo propio de scikit-learn que sugiere precios. Ambos degradan de forma explícita si fallan.

3.2 Funcionalidades excluidas explícitamente

Delimitar lo que el proyecto *no* hace es tan importante como enumerar lo que sí: evita que se evalúe contra expectativas que nunca formaron parte del alcance.

- **Cobro real de dinero.** La pasarela Stripe está integrada, pero funciona en modo de prueba con las tarjetas de ensayo del proveedor. No se procesan pagos reales.
- **Facturación electrónica ante la DIAN.** Las facturas son documentos internos con consecutivo propio; no se validan ni se transmiten a la autoridad tributaria.
- **Logística de envíos.** No hay integración con transportadoras ni seguimiento de guías: el envío asegurado se modela como un servicio más del catálogo.
- **Aplicación móvil nativa.** La interfaz es web y responsiva, verificada a 375, 768 y 1440 píxeles, pero no existe una aplicación para Android ni iOS.
- **Gestión contable.** No hay libro mayor, conciliación bancaria ni informes fiscales; el alcance llega hasta el reporte de ventas.
- **Multi-idioma y multi-moneda.** La aplicación está en español y opera en pesos colombianos.

4. Arquitectura de la solución

4.1 Visión general

La solución sigue una arquitectura de tres capas con responsabilidades separadas. El navegador ejecuta la interfaz en React; el servidor FastAPI concentra las reglas de negocio, la autenticación y la autorización; y PostgreSQL guarda la información con integridad referencial. La comunicación entre la primera y la segunda capa es HTTP con cuerpos JSON; entre la segunda y la tercera, SQL generado por SQLAlchemy.

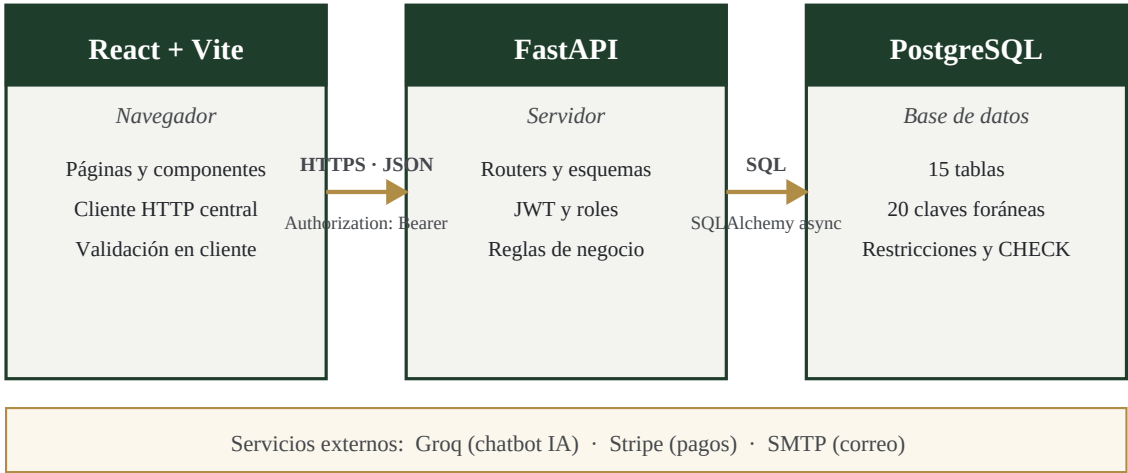


Figura 1. Arquitectura en tres capas y servicios externos.

Una decisión de diseño atraviesa todo el sistema: **la autorización definitiva la decide siempre el backend**. El frontend oculta las opciones que no corresponden al rol, pero eso es comodidad de interfaz, no seguridad. Quien llame directamente a la API sin los permisos adecuados recibe un 401 o un 403 igualmente.

4.2 Stack tecnológico

Capa	Tecnología	Versión	Por qué se eligió
Frontend	React + Vite	18 · 5.4	Componentes reutilizables y arranque de desarrollo casi instantáneo.
Estilos	Tailwind CSS	4	Sistema de diseño coherente sin mantener hojas de estilo aparte.
Enrutado	React Router DOM	6	Navegación sin recargar y rutas protegidas por rol.
Backend	FastAPI	0.141	Validación automática con Pydantic y documentación OpenAPI sin esfuerzo extra.
ORM	SQLAlchemy	2.1	Estilo declarativo moderno con tipado y soporte asíncrono.
Base de datos	PostgreSQL	16	Integridad referencial real, restricciones CHECK y agregaciones eficientes.
Autenticación	python-jose · passlib	3.5 · 1.7	JWT firmado y hashing bcrypt con sal por contraseña.
Documentos	ReportLab · openpyxl	5.0 · 3.1	Generación de facturas y reportes en PDF y Excel desde el servidor.
IA propia	scikit-learn	1.9	Regresión lineal entrenada con el propio catálogo para sugerir precios.

Capa	Tecnología	Versión	Por qué se eligió
IA externa	Groq	llama-3.3-70b	Plan gratuito suficiente para el chatbot, compatible con la API de OpenAI.
Pruebas	pytest · httpx	9.1 · 0.28	164 pruebas sobre SQLite y PostgreSQL.

4.3 Organización del código

El backend se organiza por responsabilidad, no por tipo de archivo. Cada capa solo conoce a la de abajo:

```
backend/app/  
  main.py ..... solo configuración, middlewares e include_router  
  core/ ..... ajustes, seguridad, excepciones, middlewares  
  models/ ..... tablas SQLAlchemy (15)  
  schemas/ ..... validación de entrada y salida con Pydantic  
  crud/ ..... acceso a datos y reglas de negocio  
  routers/ ..... endpoints HTTP, uno por recurso (14)  
  dependencies/ ..... autenticación, paginación y resolución de recursos  
  services/ ..... PDF, Excel, correo, chatbot, IA y tareas de fondo
```

Esa separación tiene una consecuencia práctica que conviene señalar: **la capa crud nunca lanza excepciones HTTP**. Lanza errores de dominio propios, y unos manejadores registrados en `main.py` los traducen al código HTTP que corresponde. Así la lógica de negocio no depende del protocolo y podría reutilizarse desde una tarea programada o una interfaz de línea de comandos.

5. Modelo de datos

5.1 Diagrama entidad-relación

El modelo tiene **15 tablas**, **132 columnas**, **20 claves foráneas** y **27 índices**. El diagrama muestra las entidades principales y cómo se encadenan; las tablas de apoyo (permisos, conversaciones y mensajes) se detallan en el diccionario de datos.

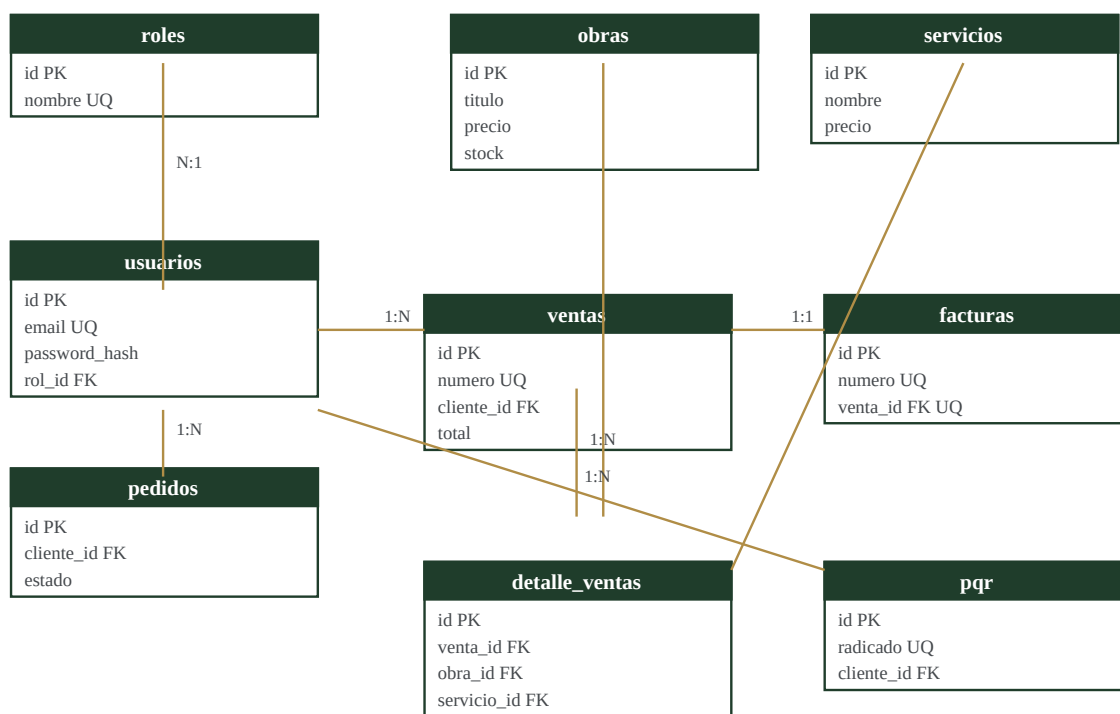


Figura 2. Diagrama entidad-relación de las entidades principales.

5.2 Decisiones de modelado que conviene explicar

- **La factura congela los datos.** Copia el nombre, el documento y la dirección del cliente, y también los importes. Si mañana el cliente cambia de dirección o sube el precio de una obra, la factura ya emitida no se altera: un documento fiscal debe reflejar lo que ocurrió, no lo que es cierto hoy.
- **Una venta se factura una sola vez.** La columna `venta_id` de la tabla `facturas` es única, así que la restricción la impone la base de datos, no solo el código.
- **Cada línea de detalle apunta a una obra o a un servicio, nunca a ambas.** Lo garantiza una restricción CHECK con un XOR, de modo que un error de programación no puede dejar datos incoherentes.
- **El dinero se guarda como NUMERIC, no como float.** Los cálculos se hacen con Decimal y redondeo ROUND_HALF_UP: con coma flotante, los céntimos acabarían descuadrando las facturas.
- **Los estados se guardan como texto con CHECK, no como ENUM nativo.** Así el mismo esquema funciona igual en PostgreSQL y en SQLite, que es lo que permite ejecutar la suite de pruebas en los dos motores.
- **Las fechas llevan zona horaria y se guardan en UTC.** La conversión a hora local se hace al presentar, nunca al almacenar.

5.3 Diccionario de datos

Las tablas siguientes se generan a partir de los modelos SQLAlchemy del proyecto, de modo que el diccionario y el código no pueden discrepar.

Tabla obras

Catálogo de obras de arte (los productos de la galería).

Columna	Tipo	Nulo	Clave	Descripción
id	INTEGER	No	PK	
titulo	VARCHAR(120)	No	—	
artista	VARCHAR(80)	No	—	
anio	INTEGER	No	—	
tecnic	VARCHAR(80)	No	—	
precio	NUMERIC(12, 2)	No	—	
descripcion	TEXT	No	—	
imagen_url	VARCHAR(255)	Sí	—	
disponible	BOOLEAN	No	—	
stock	INTEGER	No	—	
creado_en	DATETIME	No	—	Instante de creación, en UTC.
actualizado_en	DATETIME	No	—	Última modificación, en UTC.

Tabla permisos

Acciones concretas que un rol puede realizar.

Columna	Tipo	Nulo	Clave	Descripción
id	INTEGER	No	PK	
codigo	VARCHAR(60)	No	UNIQUE	
descripcion	VARCHAR(160)	No	—	

Tabla roles

Roles del sistema: administrador, empleado y cliente.

Columna	Tipo	Nulo	Clave	Descripción
id	INTEGER	No	PK	
nombre	VARCHAR(30)	No	UNIQUE	
descripcion	VARCHAR(160)	Sí	—	

Tabla servicios

Servicios: enmarcado, restauración, envío asegurado y curaduría.

Columna	Tipo	Nulo	Clave	Descripción
id	INTEGER	No	PK	
nombre	VARCHAR(100)	No	UNIQUE	
descripcion	TEXT	No	—	
precio	NUMERIC(12, 2)	No	—	
activo	BOOLEAN	No	—	
creado_en	DATETIME	No	—	Instante de creación, en UTC.
actualizado_en	DATETIME	No	—	Última modificación, en UTC.

Tabla rol_permisos

Relación muchos a muchos entre roles y permisos.

Columna	Tipo	Nulo	Clave	Descripción
rol_id	INTEGER	No	PK, FK → roles	
permiso_id	INTEGER	No	PK, FK → permisos	

Tabla usuarios

Usuarios del sistema. La contraseña solo se guarda como hash bcrypt.

Columna	Tipo	Nulo	Clave	Descripción
id	INTEGER	No	PK	
nombre	VARCHAR(40)	No	—	
apellido	VARCHAR(40)	No	—	
tipo_documento	VARCHAR(5)	No	—	
numero_documento	VARCHAR(15)	No	UNIQUE	
direccion	VARCHAR(100)	No	—	
telefono	VARCHAR(10)	No	—	
email	VARCHAR(80)	No	UNIQUE	
password_hash	VARCHAR(255)	No	—	Hash bcrypt de 60 caracteres. Nunca la contraseña en claro.
activo	BOOLEAN	No	—	
rol_id	INTEGER	No	FK → roles	
creado_en	DATETIME	No	—	Instante de creación, en UTC.
actualizado_en	DATETIME	No	—	Última modificación, en UTC.

Tabla conversaciones

Conversaciones mantenidas con el chatbot.

Columna	Tipo	Nulo	Clave	Descripción
id	INTEGER	No	PK	
session_id	VARCHAR(64)	No	—	
usuario_id	INTEGER	Sí	FK → usuarios	
titulo	VARCHAR(140)	No	—	
creado_en	DATETIME	No	—	Instante de creación, en UTC.
actualizado_en	DATETIME	No	—	Última modificación, en UTC.

Tabla pedidos

Órdenes que el cliente arma desde el sitio web.

Columna	Tipo	Nulo	Clave	Descripción
id	INTEGER	No	PK	
cliente_id	INTEGER	No	FK → usuarios	
estado	VARCHAR(20)	No	—	
total	NUMERIC(12, 2)	No	—	
creado_en	DATETIME	No	—	Instante de creación, en UTC.
actualizado_en	DATETIME	No	—	Última modificación, en UTC.

Tabla pqr

Peticiones, quejas, reclamos y sugerencias.

Columna	Tipo	Nulo	Clave	Descripción
id	INTEGER	No	PK	
radicado	VARCHAR(24)	No	UNIQUE	Consecutivo que asigna el servidor.
cliente_id	INTEGER	Sí	FK → usuarios	
contacto_nombre	VARCHAR(90)	No	—	
contacto_email	VARCHAR(80)	No	—	
tipo	VARCHAR(20)	No	—	
asunto	VARCHAR(140)	No	—	
mensaje	TEXT	No	—	
estado	VARCHAR(20)	No	—	
respuesta	TEXT	Sí	—	
respondido_por_id	INTEGER	Sí	FK → usuarios	
respondido_en	DATETIME	Sí	—	
creado_en	DATETIME	No	—	Instante de creación, en UTC.
actualizado_en	DATETIME	No	—	Última modificación, en UTC.

Tabla detalles_pedido

Líneas de un pedido: una obra o un servicio por línea.

Columna	Tipo	Nulo	Clave	Descripción
id	INTEGER	No	PK	
pedido_id	INTEGER	No	FK → pedidos	
obra_id	INTEGER	Sí	FK → obras	
servicio_id	INTEGER	Sí	FK → servicios	
descripcion	VARCHAR(160)	No	—	
cantidad	INTEGER	No	—	
precio_unitario	NUMERIC(12, 2)	No	—	

Tabla mensajes

Mensajes individuales de cada conversación.

Columna	Tipo	Nulo	Clave	Descripción
id	INTEGER	No	PK	
conversacion_id	INTEGER	No	FK → conversaciones	
rol	VARCHAR(20)	No	—	
contenido	TEXT	No	—	
creado_en	DATETIME	No	—	Instante de creación, en UTC.
actualizado_en	DATETIME	No	—	Última modificación, en UTC.

Tabla ventas

Transacciones comerciales con su desglose económico completo.

Columna	Tipo	Nulo	Clave	Descripción
id	INTEGER	No	PK	
numero	VARCHAR(24)	No	UNIQUE	Consecutivo que asigna el servidor.
cliente_id	INTEGER	No	FK → usuarios	
usuario_id	INTEGER	Sí	FK → usuarios	
pedido_id	INTEGER	Sí	FK → pedidos, UNIQUE	
estado	VARCHAR(20)	No	—	
metodo_pago	VARCHAR(20)	No	—	
subtotal	NUMERIC(12, 2)	No	—	
descuento	NUMERIC(12, 2)	No	—	
impuestos	NUMERIC(12, 2)	No	—	
total	NUMERIC(12, 2)	No	—	
observaciones	TEXT	Sí	—	
creado_en	DATETIME	No	—	Instante de creación, en UTC.

Columna	Tipo	Nulo	Clave	Descripción
actualizado_en	DATETIME	No	—	Última modificación, en UTC.

Tabla detalle_ventas

Líneas de una venta, con precio y descuento congelados.

Columna	Tipo	Nulo	Clave	Descripción
id	INTEGER	No	PK	
venta_id	INTEGER	No	FK → ventas	
obra_id	INTEGER	Sí	FK → obras	
servicio_id	INTEGER	Sí	FK → servicios	
descripcion	VARCHAR(160)	No	—	
cantidad	INTEGER	No	—	
precio_unitario	NUMERIC(12, 2)	No	—	
descuento	NUMERIC(12, 2)	No	—	
subtotal	NUMERIC(12, 2)	No	—	

Tabla facturas

Documentos fiscales emitidos a partir de una venta.

Columna	Tipo	Nulo	Clave	Descripción
id	INTEGER	No	PK	
numero	VARCHAR(24)	No	UNIQUE	Consecutivo que asigna el servidor.
venta_id	INTEGER	No	FK → ventas, UNIQUE	
cliente_id	INTEGER	No	FK → usuarios	
estado	VARCHAR(20)	No	—	
fecha_emision	DATETIME	No	—	
cliente_nombre	VARCHAR(90)	No	—	
cliente_documento	VARCHAR(25)	No	—	
cliente_email	VARCHAR(80)	No	—	
cliente_direccion	VARCHAR(100)	No	—	
cliente_telefono	VARCHAR(15)	No	—	
subtotal	NUMERIC(12, 2)	No	—	
descuento	NUMERIC(12, 2)	No	—	
impuestos	NUMERIC(12, 2)	No	—	
iva_porcentaje	NUMERIC(5, 2)	No	—	
total	NUMERIC(12, 2)	No	—	
observaciones	TEXT	Sí	—	
creado_en	DATETIME	No	—	Instante de creación, en UTC.

Columna	Tipo	Nulo	Clave	Descripción
actualizado_en	DATETIME	No	—	Última modificación, en UTC.

Tabla pagos

Pagos con Stripe. Solo referencias; ningún dato de tarjeta.

Columna	Tipo	Nulo	Clave	Descripción
id	INTEGER	No	PK	
venta_id	INTEGER	No	FK → ventas	
proveedor	VARCHAR(30)	No	—	
referencia_externa	VARCHAR(255)	Sí	—	
payment_intent	VARCHAR(255)	Sí	—	
estado	VARCHAR(20)	No	—	
monto	NUMERIC(12, 2)	No	—	
moneda	VARCHAR(6)	No	—	
detalle	TEXT	Sí	—	
creado_en	DATETIME	No	—	Instante de creación, en UTC.
actualizado_en	DATETIME	No	—	Última modificación, en UTC.

6. Diseño de la solución

6.1 Historias de usuario y su implementación

Cada historia se enuncia desde la necesidad de quien la usa, y se indica dónde quedó implementada para que pueda verificarse.

#	Historia de usuario	Dónde está implementada
HU-01	Como visitante quiero recorrer el catálogo y filtrar por artista, técnica y precio, para encontrar una obra que me interese.	GET /api/productos con seis filtros y paginación · página Catálogo
HU-02	Como visitante quiero crear una cuenta validando mis datos, para poder comprar.	POST /api/auth/registro · RegisterModal.jsx · doble validación
HU-03	Como cliente quiero armar un pedido con varias obras y servicios, para comprarlos juntos.	POST /api/pedidos · pedidos y detalles_pedido
HU-04	Como cliente quiero consultar mis compras y descargar la factura, para tener el soporte.	GET /api/facturas/{id}/pdf · panel del cliente
HU-05	Como cliente quiero radicar una PQR y seguir su estado, para saber si me van a responder.	POST /api/pqr · radicado y cuatro estados
HU-06	Como empleado quiero registrar una venta presencial, para que quede en el sistema aunque no venga del sitio web.	POST /api/ventas · panel de ventas
HU-07	Como empleado quiero emitir la factura de una venta, para entregarle el documento al cliente.	POST /api/facturas · consecutivo OL-NNNNNN
HU-08	Como administrador quiero ver cuánto se vendió en un periodo y qué se vendió más, para tomar decisiones.	GET /api/dashboard · 12 indicadores y 4 gráficos
HU-09	Como administrador quiero descargar el reporte diario en Excel, para seguir analizándolo por mi cuenta.	GET /api/reportes/ventas-diarias/excel
HU-10	Como administrador quiero activar o desactivar un usuario sin borrarlo, para conservar su histórico.	PATCH /api/usuarios/{id}/estado
HU-11	Como visitante quiero preguntar por las obras a cualquier hora y que me respondan con información real.	POST /api/chatbot/mensaje · Groq con el catálogo como contexto

6.2 El flujo comercial, paso a paso

El encadenamiento pedido → venta → factura es la columna vertebral del sistema y conviene entenderlo entero, porque cada paso deja rastro en una tabla distinta:

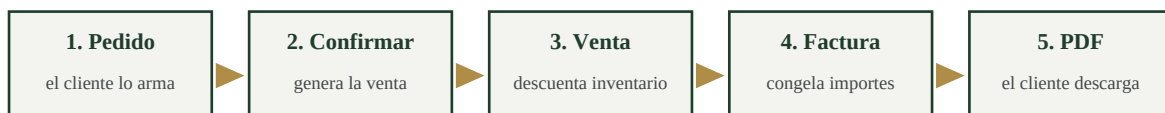


Figura 3. Encadenamiento del flujo comercial.

Confirmar un pedido es la operación más delicada del sistema: **crea la venta, copia sus líneas, calcula el IVA y descuenta el inventario, todo dentro de una única transacción.** Si cualquiera de esos pasos falla, se deshacen todos. Sin esa garantía podría quedar una venta sin líneas, o inventario descontado sin venta.

6.3 Máquinas de estado

Los estados no cambian libremente: cada transición se valida contra una tabla de transiciones permitidas. Una venta pagada, por ejemplo, **no puede anularse**: debe reembolsarse, porque eso sí deja rastro contable.

Entidad	Estado	Transiciones permitidas
Pedido	pendiente	confirmado · cancelado
	confirmado	entregado · cancelado
	entregado / cancelado	— (final)
Venta	pendiente_pago	pagada · anulada
	pagada	reembolsada
	anulada / reembolsada	— (final)
Factura	emitida	pagada · anulada
	pagada	anulada
PQR	pendiente	en_proceso · respondida · cerrada
	respondida	cerrada · en_proceso

7. Manual de instalación y configuración

7.1 Requisitos previos

Componente	Versión mínima	Cómo comprobarlo
Python	3.11 (probado en 3.12)	<code>python --version</code>
Node.js	18 (probado en 20)	<code>node --version</code>
PostgreSQL	14 (probado en 16)	<code>psql --version</code>
Git	cualquiera reciente	<code>git --version</code>

7.2 Paso 1 — Obtener el código

```
git clone https://github.com/vegajerson88-droid/Oleo-Lienzo.git
cd Oleo-Lienzo
```

7.3 Paso 2 — Crear la base de datos

```
sudo -u postgres psql
CREATE ROLE oleo WITH LOGIN PASSWORD 'una_contrasena_segura';
CREATE DATABASE oleo_lienzo WITH OWNER = oleo ENCODING = 'UTF8';
\q
```

El esquema puede crearse de dos maneras, y ambas producen exactamente las mismas tablas: ejecutando el script SQL incluido, o dejando que la aplicación lo cree al arrancar. El script existe porque es un entregable exigido y porque permite revisar el esquema sin ejecutar nada.

```
psql -U oleo -d oleo_lienzo -f backend/sql/schema_postgresql.sql
```

7.4 Paso 3 — Configurar el backend

```
cd backend
python -m venv venv
source venv/bin/activate # en Windows: venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env # y editarlo (ver la tabla de abajo)
```

7.5 Variables de entorno del backend

El archivo `.env` **no se versiona**: está en el `.gitignore` y solo se sube la plantilla `.env.example`. Ninguna credencial aparece en el código fuente.

Variable	Obligatoria	Para qué sirve
DATABASE_URL	Sí	Conexión a PostgreSQL. Formato: <code>postgresql+psycopg://usuario:clave@host:5432/base</code>
JWT_SECRET_KEY	Sí en producción	Clave de firma de los tokens. No tiene valor por defecto en el código. En desarrollo se genera una efímera; en producción, si falta, la aplicación se niega a arrancar.
CORS_ORIGINS	Sí	Lista explícita de orígenes permitidos, separados por comas. No se acepta el comodín <code>*</code> .
GROQ_API_KEY	No	Clave del chatbot con IA. Si falta, el chatbot responde en modo local y lo declara abiertamente.
STRIPE_SECRET_KEY	No	Pasarela de pago en modo de prueba. Si falta, el módulo informa de que no está configurado.

Variable	Obligatoria	Para qué sirve
EMAIL_HOST y afines	No	Servidor SMTP. Si falta, los correos se registran en el log en vez de enviarse.

Para generar una clave de firma segura:

```
python -c "import secrets; print(secrets.token_urlsafe(48))"
```

7.6 Paso 4 — Poblar la base y arrancar el backend

```
python seed.py # datos de ejemplo y entrenamiento del modelo de IA
uvicorn app.main:app --reload --port 8000
```

El script `seed.py` es idempotente: puede ejecutarse varias veces sin duplicar registros. Crea los permisos, los tres roles, cuatro usuarios de prueba, diez obras, cuatro servicios, ventas, facturas, pedidos y PQR de ejemplo, y entrena el modelo de sugerencia de precios.

7.7 Paso 5 — Configurar y arrancar el frontend

```
cd frontend
npm install
cp .env.example .env # ajustar VITE_API_URL si el backend no está en :8000
npm run dev
```

7.8 Comprobación de que todo quedó bien

Qué comprobar	Dónde	Qué se debe ver
La API responde	<code>http://localhost:8000/api/sistema/salud</code>	<code>{"estado":"ok"}</code>
La documentación	<code>http://localhost:8000/docs</code>	Swagger con 60 operaciones
El sitio web	<code>http://localhost:5173</code>	La galería con el carrusel
El inicio de sesión	<code>/login</code>	Entrar con <code>admin@oleoylienzo.com</code> / <code>Admin1234</code>
Las pruebas	<code>cd backend && pytest -q</code>	164 passed

7.9 Credenciales de prueba

Las crea `seed.py` y solo sirven en entorno de desarrollo. **Deben cambiarse antes de cualquier despliegue real.**

Rol	Correo	Contraseña
Administrador	<code>admin@oleoylienzo.com</code>	<code>Admin1234</code>
Empleado	<code>empleado@oleoylienzo.com</code>	<code>Empleado123</code>
Cliente	<code>cliente@oleoylienzo.com</code>	<code>Cliente123</code>
Cliente	<code>carlos.mejia@ejemplo.com</code>	<code>Cliente123</code>

8. Documentación técnica de la API

La API expone **60 operaciones** repartidas en 14 recursos, todas bajo el prefijo /api. La documentación interactiva se genera sola y está disponible en /docs (Swagger UI) y /redoc.

8.1 Cómo autenticarse

Las rutas protegidas esperan la cabecera Authorization: Bearer <token>. El token se obtiene del inicio de sesión y caduca a los 60 minutos.

```
# 1. Obtener el token
curl -X POST http://localhost:8000/api/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"email":"admin@oleoylienzo.com","password":"Admin1234"}'

# Respuesta
{ "access_token": "eyJhbGciOiJIUzI1NiIs... ", "token_type": "bearer",
  "usuario": { "id": 1, "nombre": "Ana", "rol": "administrador" } }

# 2. Usarlo en una ruta protegida
curl http://localhost:8000/api/usuarios \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIs... '
```

8.2 Ejemplo completo: registrar una venta

```
POST /api/ventas
Authorization: Bearer <token de administrador o empleado>

{
  "cliente_id": 3,
  "detalles": [
    { "obra_id": 1, "cantidad": 1, "descuento": 0 },
    { "servicio_id": 1, "cantidad": 1 }
  ],
  "metodo_pago": "tarjeta"
}

201 Created
{
  "id": 7, "numero": "V-2026-000007", "estado": "pendiente_pago",
  "subtotal": 1400000.00, "impuestos": 266000.00, "total": 1666000.00,
  "detalles": [ ... ]
}
```

Obsérvese que el cliente **no envía** ni el número de venta, ni el estado, ni los importes: todo eso lo calcula y lo asigna el servidor. Los esquemas de entrada ni siquiera aceptan esos campos.

8.3 Formato uniforme de los errores

Todos los errores de la API, sin excepción, comparten el mismo cuerpo. Eso permite al frontend tratarlos con un único bloque de código:

```
404 Not Found
{ "error": "NotFound", "detail": "Obra 999 no encontrada." }

422 Unprocessable Content
{ "error": "ValidationError",
  "detail": [ { "loc": ["query","page"], "msg": "Input should be >= 1" } ] }

409 Conflict
{ "error": "Conflict", "detail": "Ya existe un usuario con ese correo." }
```

Además, cada respuesta incluye la cabecera X-Request-ID, que aparece también en el registro del servidor: permite localizar en el log exactamente la petición que falló.

8.4 Catálogo completo de operaciones

Autenticación

Verbo	Ruta	Operación
POST	/api/auth/cambiar-password	Cambiar la contraseña con la sesión iniciada
POST	/api/auth/login	Iniciar sesión y obtener el JWT
GET	/api/auth/me	Datos del usuario autenticado
POST	/api/auth/recuperar-password	Solicitar el enlace de recuperación de contraseña
POST	/api/auth/registro	Registrar un cliente nuevo
POST	/api/auth/restablecer-password	Fijar una contraseña nueva con el token del correo
POST	/api/auth/token	Inicio de sesión compatible con OAuth2 (botón Authorize de Swagger)

Usuarios

Verbo	Ruta	Operación
GET	/api/usuarios	Listar usuarios
POST	/api/usuarios	Crear un usuario desde el panel
GET	/api/usuarios/roles	Listar roles y sus permisos
DELETE	/api/usuarios/{usuario_id}	Eliminar un usuario
GET	/api/usuarios/{usuario_id}	Consultar un usuario
PATCH	/api/usuarios/{usuario_id}	Actualizar un usuario (parcialmente)
PUT	/api/usuarios/{usuario_id}	Reemplazar un usuario (actualización completa)
PATCH	/api/usuarios/{usuario_id}/estado	Activar o desactivar un usuario

Obras / Productos

Verbo	Ruta	Operación
GET	/api/productos	Listar obras del catálogo
POST	/api/productos	Crear una obra
DELETE	/api/productos/{obra_id}	Eliminar una obra
GET	/api/productos/{obra_id}	Consultar una obra
PATCH	/api/productos/{obra_id}	Actualizar una obra (parcialmente)
PUT	/api/productos/{obra_id}	Reemplazar una obra (actualización completa)

Servicios

Verbo	Ruta	Operación
GET	/api/servicios	Listar servicios
POST	/api/servicios	Crear un servicio
DELETE	/api/servicios/{servicio_id}	Eliminar un servicio
GET	/api/servicios/{servicio_id}	Consultar un servicio
PATCH	/api/servicios/{servicio_id}	Actualizar un servicio (parcialmente)
PUT	/api/servicios/{servicio_id}	Reemplazar un servicio (actualización completa)

Pedidos

Verbo	Ruta	Operación
GET	/api/pedidos	Listar pedidos
POST	/api/pedidos	Crear un pedido
GET	/api/pedidos/{pedido_id}	Consultar un pedido
PATCH	/api/pedidos/{pedido_id}/estado	Cambiar el estado de un pedido

Ventas

Verbo	Ruta	Operación
GET	/api/ventas	Historial de ventas con filtros
POST	/api/ventas	Registrar una venta
GET	/api/ventas/{venta_id}	Consultar una venta
PATCH	/api/ventas/{venta_id}/estado	Cambiar el estado de una venta

Facturación

Verbo	Ruta	Operación
GET	/api/facturas	Consultar facturas
POST	/api/facturas	Emitir una factura a partir de una venta
GET	/api/facturas/{factura_id}	Consultar una factura
PATCH	/api/facturas/{factura_id}/estado	Cambiar el estado de una factura
GET	/api/facturas/{factura_id}/pdf	Descargar la factura en PDF

Reportes

Verbo	Ruta	Operación
GET	/api/reportes/ventas-diarias	Reporte diario de ventas (JSON)
GET	/api/reportes/ventas-diarias/excel	Reporte diario de ventas en Excel (.xlsx)
GET	/api/reportes/ventas-diarias/pdf	Reporte diario de ventas en PDF

Dashboard

Verbo	Ruta	Operación
GET	/api/dashboard	Dashboard del usuario autenticado

PQR

Verbo	Ruta	Operación
GET	/api/pqr	Listar PQR
POST	/api/pqr	Radical una PQR
GET	/api/pqr/{pqr_id}	Consultar una PQR
PATCH	/api/pqr/{pqr_id}/estado	Cambiar el estado de una PQR
POST	/api/pqr/{pqr_id}/responder	Responder una PQR

Chatbot IA

Verbo	Ruta	Operación
GET	/api/chatbot/conversaciones	Listar conversaciones (auditoría)
POST	/api/chatbot/mensaje	Conversar con el asistente
GET	/api/chatbot/conversaciones/{conversacion_id}	Consultar una conversación completa

Pagos (Stripe)

Verbo	Ruta	Operación
POST	/api/pagos/checkout	Crear la sesión de pago de una venta
GET	/api/pagos/configuracion	Clave pública de Stripe
POST	/api/pagos/webhook	Webhook de Stripe

Inteligencia Artificial

Verbo	Ruta	Operación
GET	/api/ia/descripcion-sugerida	Redactar una descripción con IA externa
GET	/api/ia/precio-sugerido	Sugerir un precio con el modelo propio

Sistema

Verbo	Ruta	Operación
GET	/	Información de la API
GET	/api/sistema/diagnostico	Diagnóstico completo de dependencias
GET	/api/sistema/salud	Comprobación rápida de vida

9. Manual de usuario

Esta sección recorre la aplicación desde la perspectiva de quien la usa. Las capturas se tomaron de la aplicación en funcionamiento con los datos de ejemplo que genera seed.py.

9.1 Zona pública

Cualquiera puede recorrer la galería sin registrarse. La página de inicio presenta el carrusel con las diez obras de la colección; cada una muestra su título, su autor y su ficha técnica.

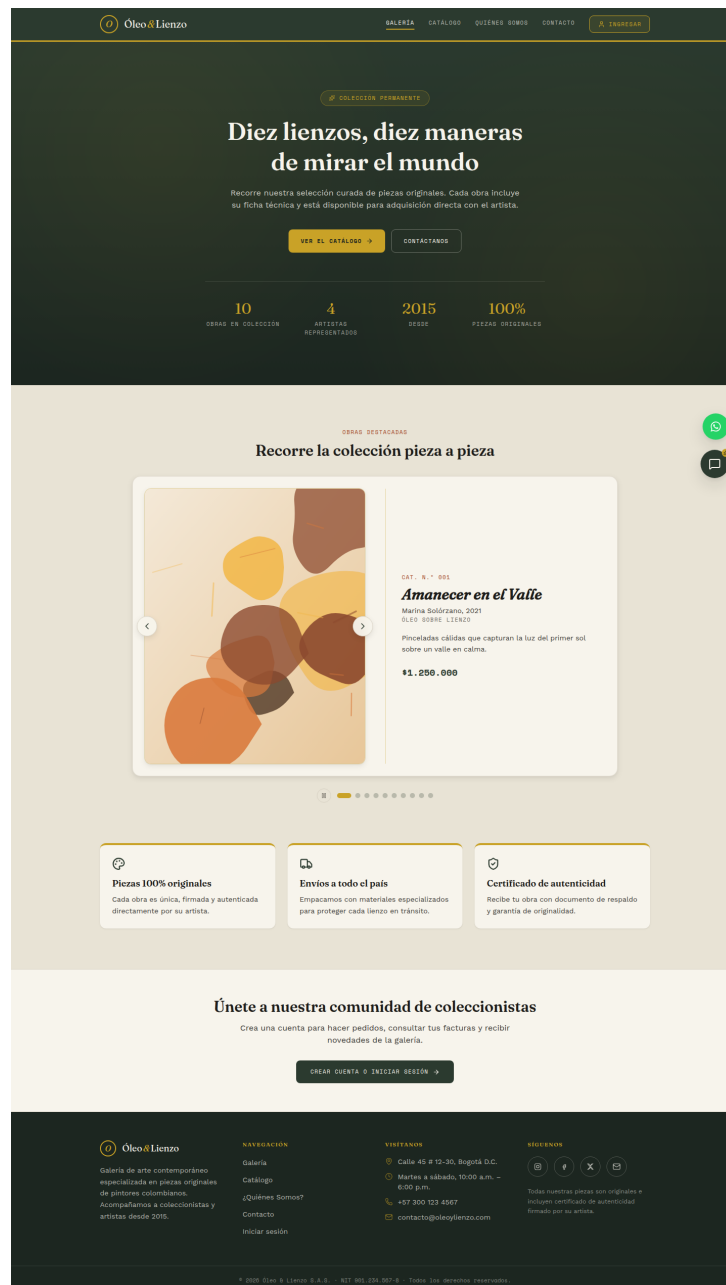


Figura 1. Página de inicio con el carrusel de obras.

El catálogo permite filtrar por artista, técnica, disponibilidad y rango de precio, y buscar por texto. Los resultados se paginan.

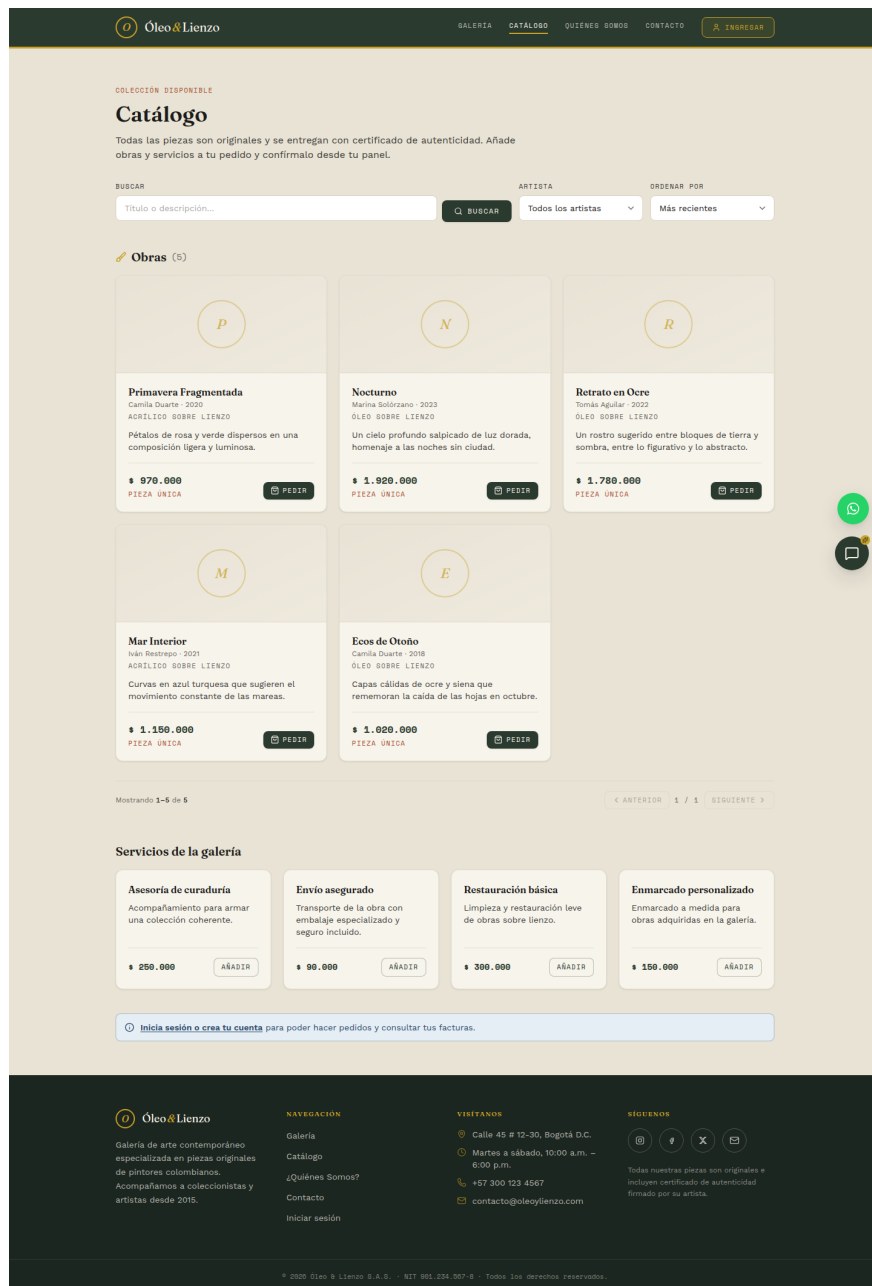


Figura 2. Catálogo con filtros y paginación.

HABLEMOS

Contacto y PQR

¿Te interesa una pieza, quieres agendar una visita o necesitas presentar una petición, queja o reclamo? Escríbenos y te damos un número de radicado para hacer seguimiento.

TIPO DE SOLICITUD *

Petición

NOMBRE *

CORREO ELECTRÓNICO *

ASUNTO *

Resume tu solicitud en una línea

0/140

MENSAJE *

Cuéntanos con detalle qué necesitas...

0/2000

ENVIAR SOLICITUD

Visítanos

DIRECCIÓN
Calle 45 # 12-30, Bogotá D.C.

HORARIO
Martes a sábado, 10:00 a.m. - 6:00 p.m.

TELÉFONO
+57 300 123 4567

CORREO
contacto@oleoylienzo.com

También puedes escribirnos por WhatsApp o preguntarle al asistente virtual, con los botones de la esquina inferior derecha.

Oleo & Lienzo

Galería de arte contemporáneo especializada en piezas originales de pintores colombianos. Acompañamos a coleccionistas y artistas desde 2015.

NAVEGACIÓN

Galería

Catálogo

¿Quiénes Somos?

Contacto

Iniciar sesión

VISITANOS

Calle 45 # 12-30, Bogotá D.C.

Martes a sábado, 10:00 a.m. - 6:00 p.m.

+57 300 123 4567

contacto@oleoylienzo.com

SÍGUENOS

Todas nuestras piezas son originales e incluyen certificado de autenticidad firmado por su artista.

© 2020 Oleo & Lienzo S.A.S. · NIT 901.234.067-8 · Todos los derechos reservados.

Figura 3. Formulario de contacto con validación en tiempo real.

9.2 Registro e inicio de sesión

El formulario de inicio de sesión valida mientras se escribe. El registro se abre en una ventana modal que puede cerrarse sin completar el proceso, con la tecla Escape, haciendo clic fuera o con el botón Cancelar.

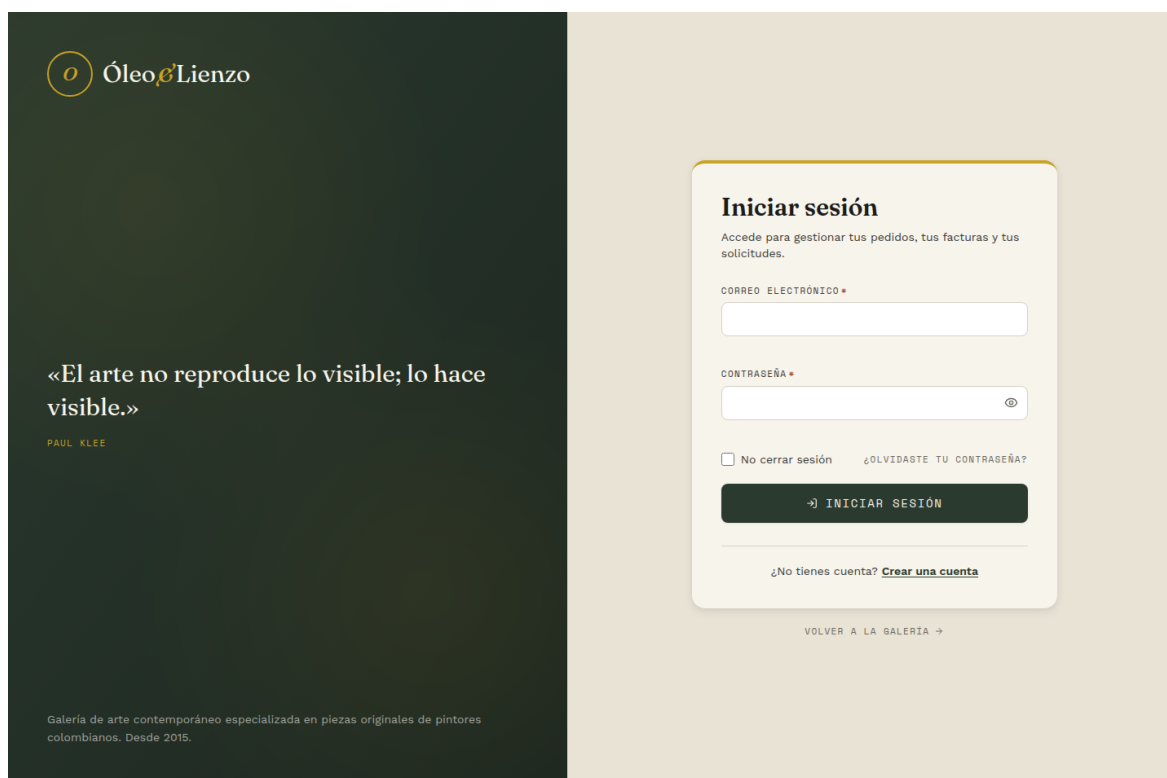


Figura 4. Inicio de sesión con validación en tiempo real.

Tras entrar, el nombre de la persona aparece en la barra de navegación junto al botón de salir, y el menú se adapta al rol.

9.3 Panel del administrador

El dashboard reúne doce indicadores y cuatro gráficos, todos calculados en la base de datos. Los filtros de la parte superior —fechas, estado, obra, servicio y cliente— afectan a la vez a las tarjetas y a los gráficos.

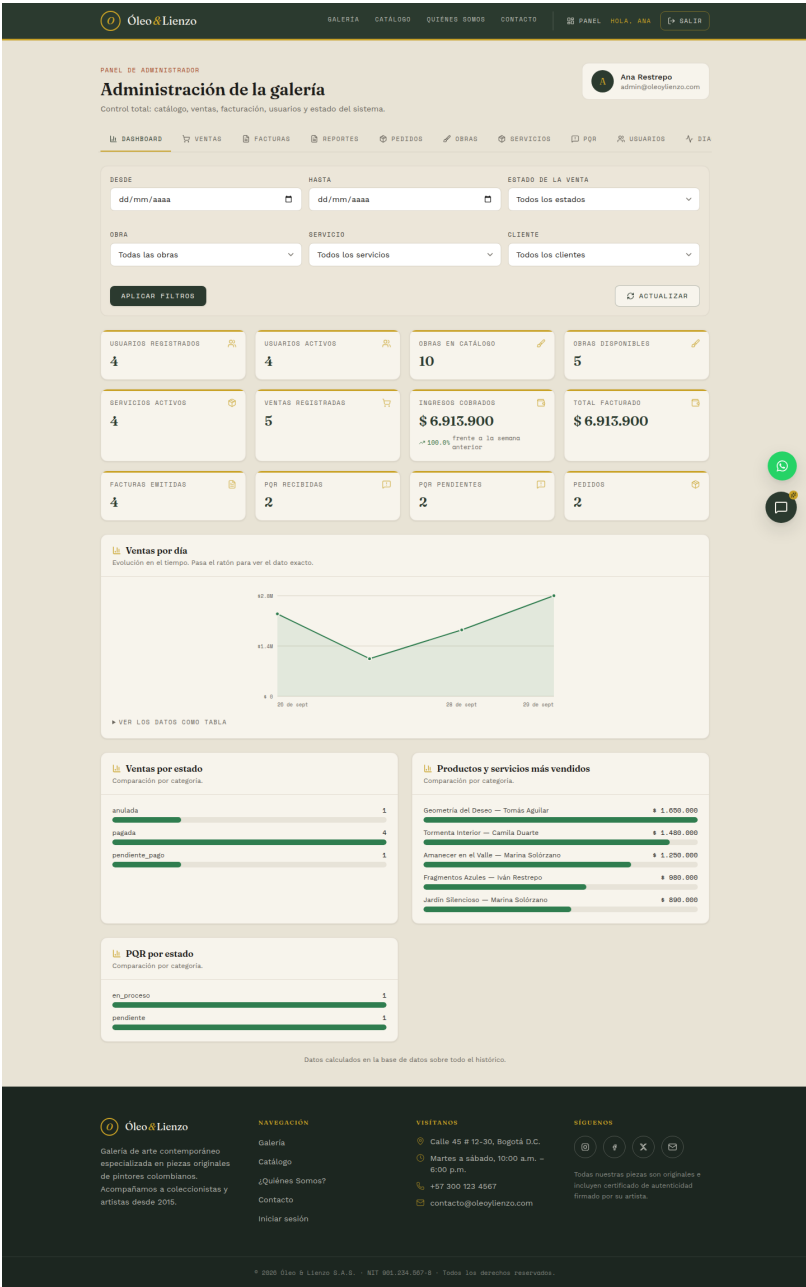


Figura 5. Dashboard del administrador con filtros, indicadores y gráficos.

El historial de ventas permite consultar por fecha, cliente, obra, servicio, estado y rango de valor, y cambiar el estado de cada venta respetando las transiciones válidas.

Figura 6. Historial de ventas con filtros y desglose de IVA.

Página 29

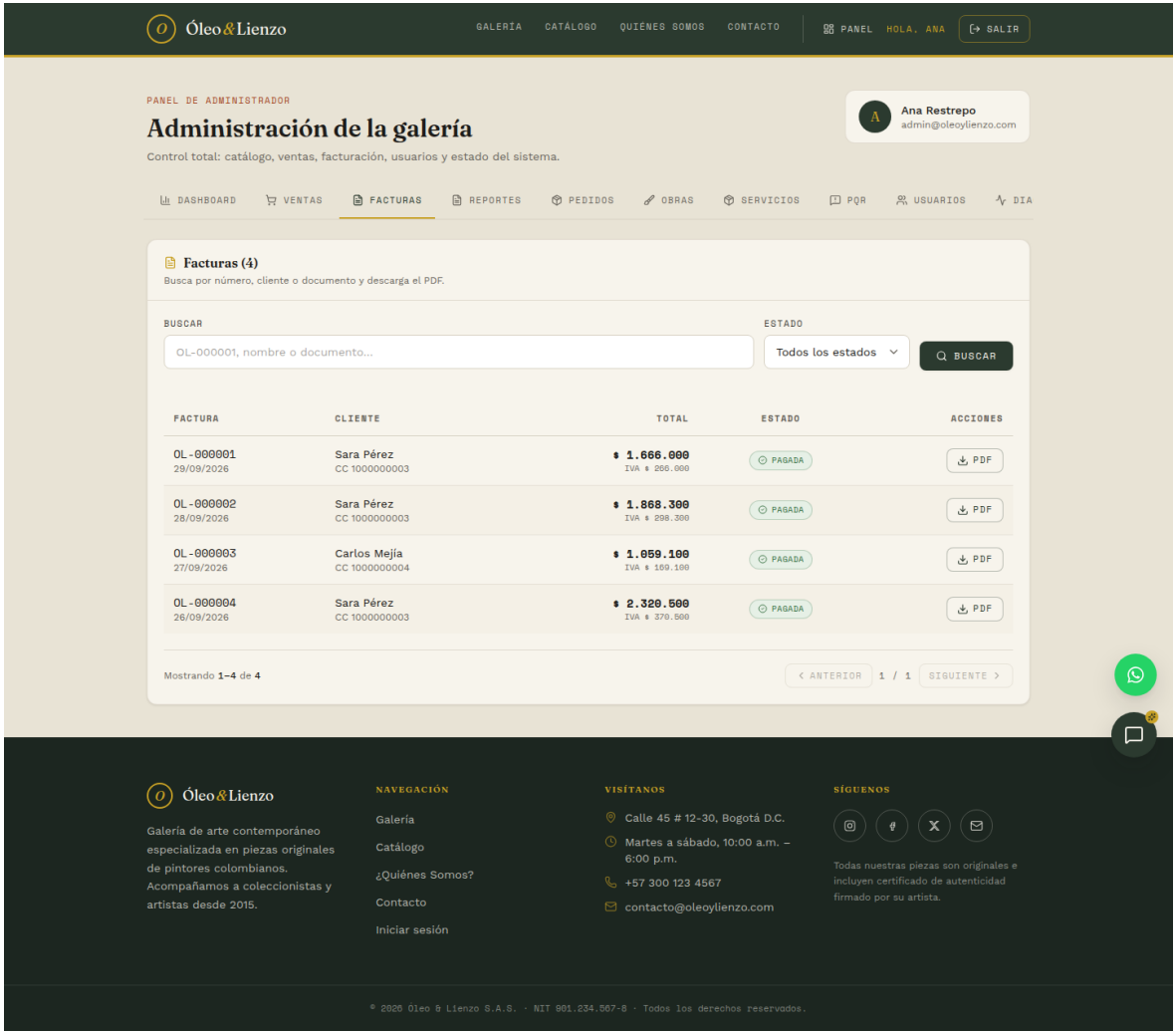


Figura 7. Facturación: consulta y descarga del PDF.

El reporte diario de ventas se consulta en pantalla y se exporta en PDF o en Excel. El archivo de Excel incluye fórmulas y autofiltro para seguir analizando los datos.

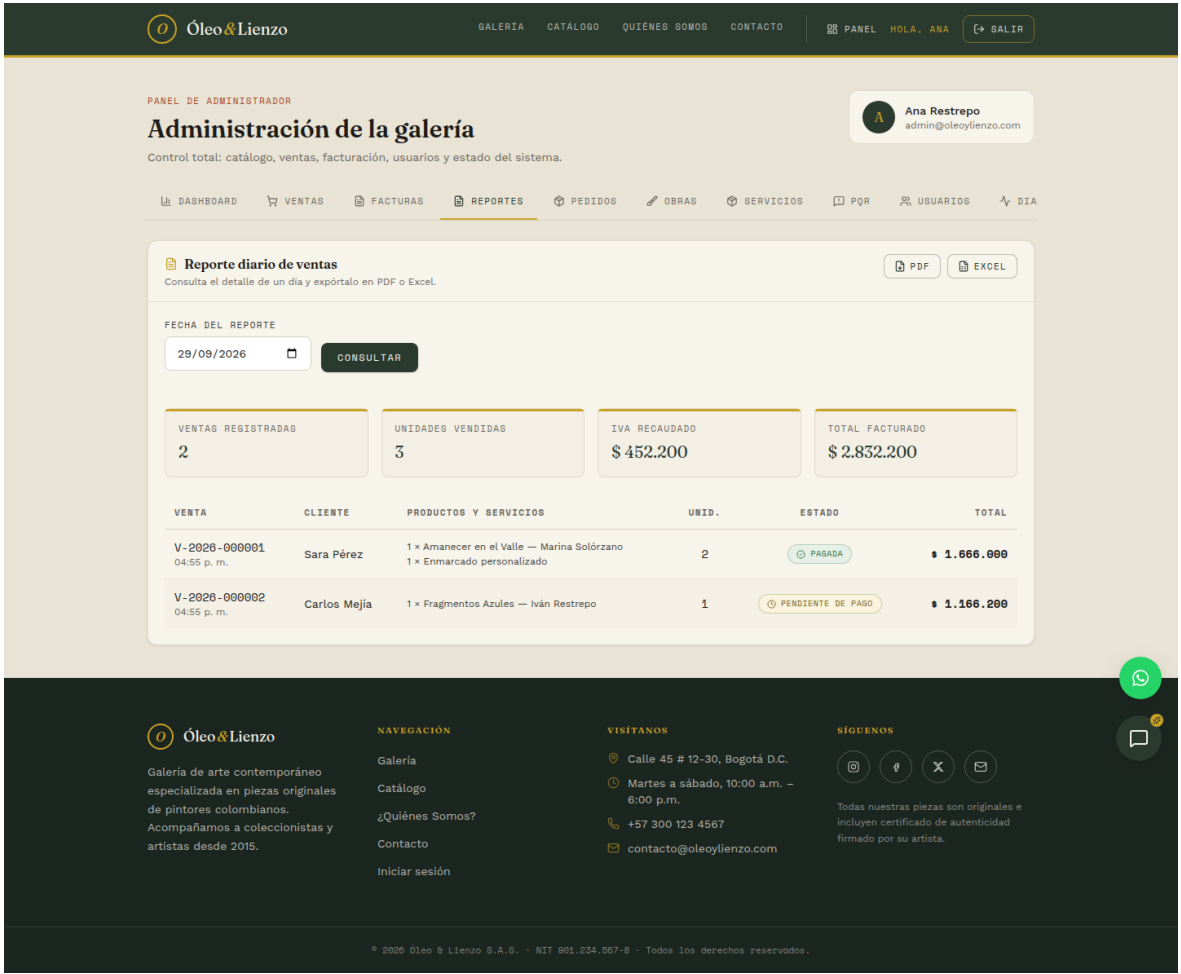


Figura 8. Reporte diario con exportación a PDF y Excel.

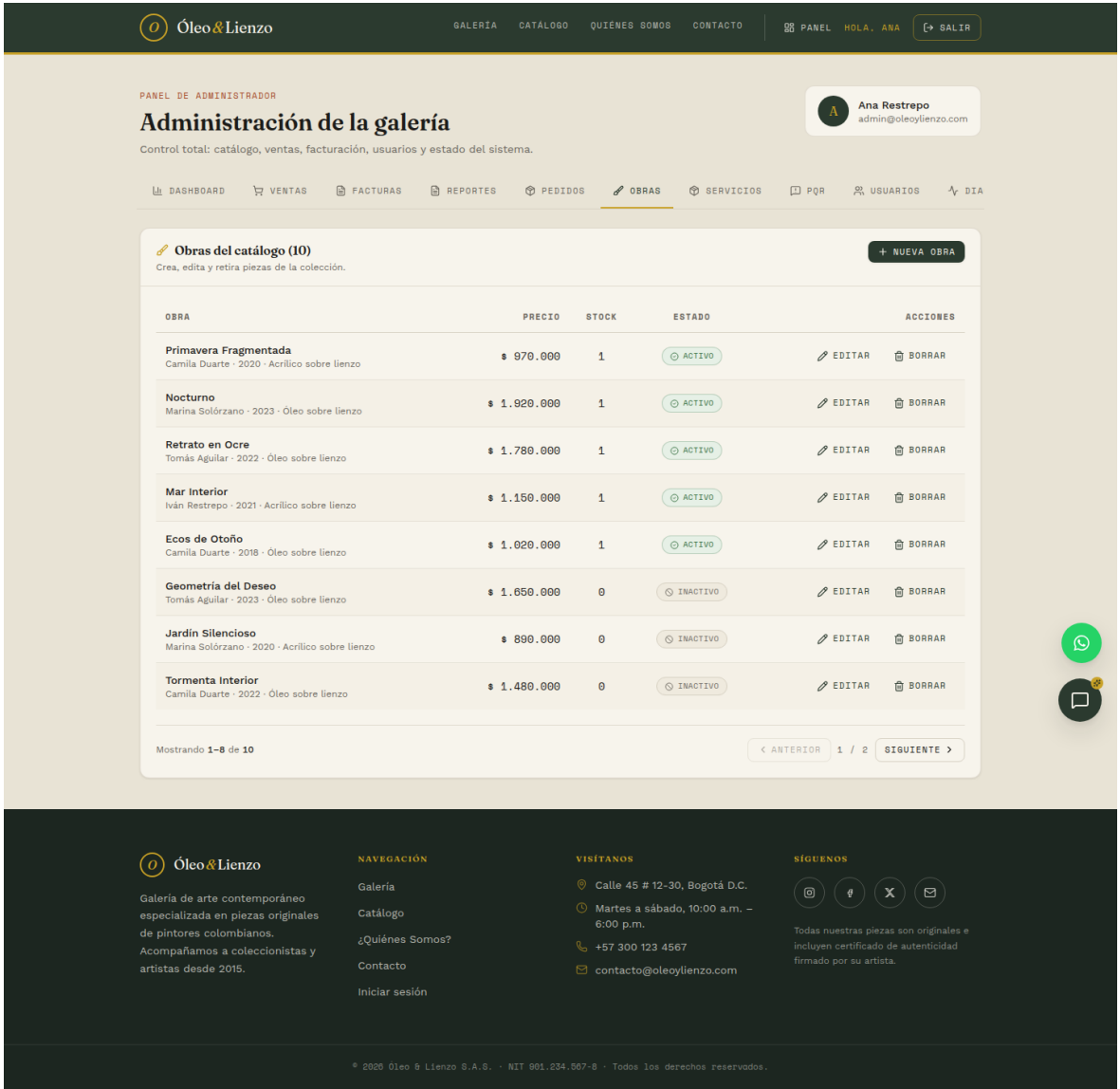


Figura 9. Gestión del catálogo, con sugerencia de precio por IA.

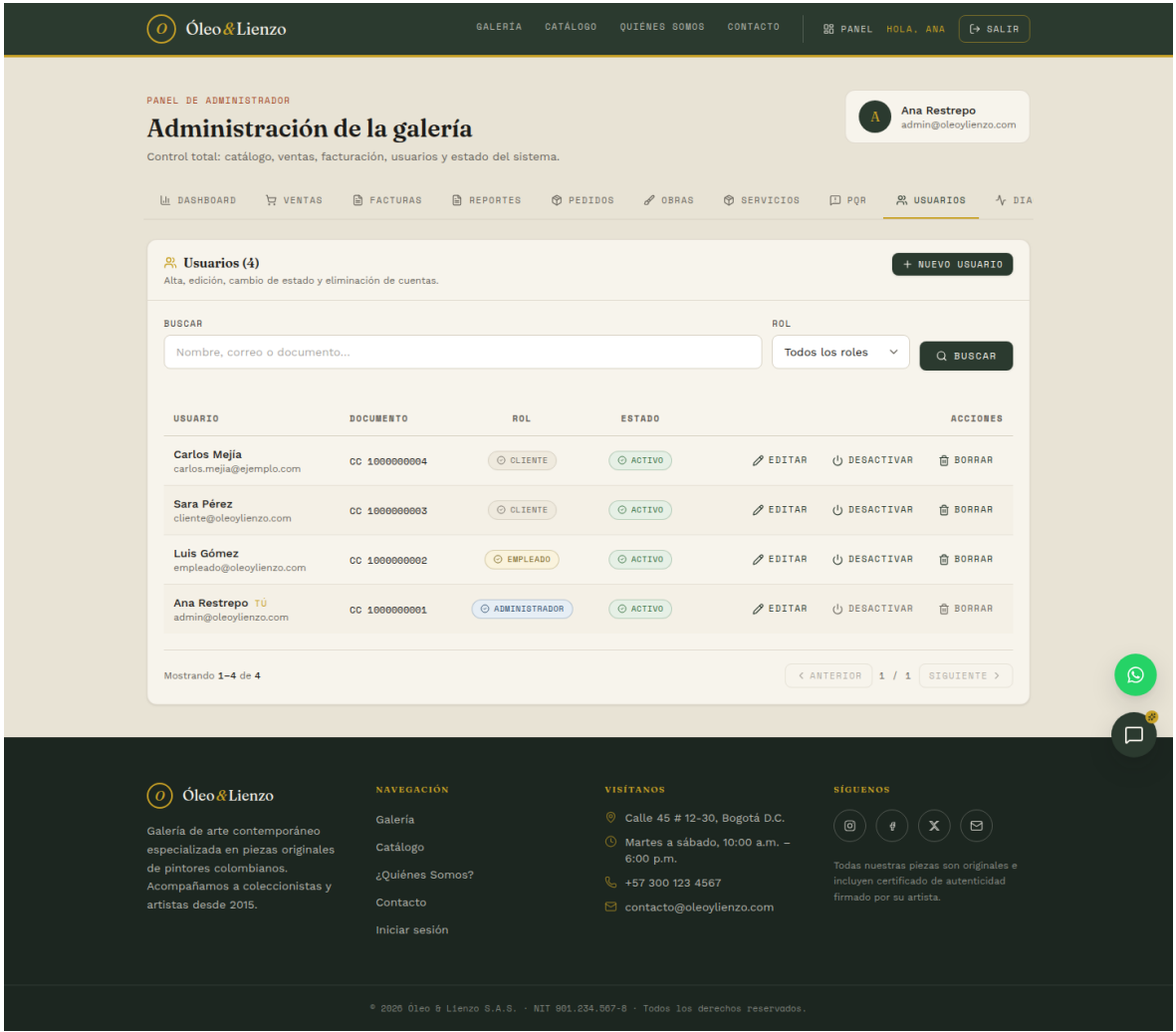


Figura 10. Gestión de usuarios, roles y estado.

Óleo & Lienzo

GALERIA CATALOGO QUIENES SOMOS CONTACTO

PANEL DE ADMINISTRADOR

Administración de la galería

Control total: catálogo, ventas, facturación, usuarios y estado del sistema.

ANS RESTREPO admin@oleoylienzo.com

DASHBOARD VENTAS FACTURAS REPORTES PEDIDOS OBRAS SERVICIOS **PQR** USUARIOS DIA

PQR (2)

Peticiones, quejas, reclamos y sugerencias de los clientes.

Todos los estac Todos los tipos

PQR-000002 QUEJA EN PROCESO

Demora en la entrega

Sara Pérez · cliente@oleoylienzo.com · 29/09/2026

El pedido tardó más de lo indicado en llegar a mi dirección.

RESPONDER CERRAR

PQR-000001 PETICIÓN PENDIENTE

Solicitud de certificado de autenticidad

Sara Pérez · cliente@oleoylienzo.com · 29/09/2026

Quisiera recibir de nuevo el certificado de autenticidad de mi última compra.

RESPONDER PONER EN PROCESO CERRAR

Mostrando 1-2 de 2

ANTERIOR 1 / 1 SIGUIENTE

Óleo & Lienzo

Galería de arte contemporáneo especializada en piezas originales de pintores colombianos. Acompañamos a coleccionistas y artistas desde 2015.

NAVEGACIÓN

Galería

Catálogo

¿Quiénes Somos?

Contacto

Iniciar sesión

VISÍTANOS

Calle 45 # 12-30, Bogotá D.C.

Martes a sábado, 10:00 a.m. – 6:00 p.m.

+57 300 123 4567

contacto@oleoylienzo.com

SÍGUENOS

Todas nuestras piezas son originales e incluyen certificado de autenticidad firmado por su artista.

© 2026 Óleo & Lienzo S.A.S. · NIT 901.234.567-8 · Todos los derechos reservados.

Figura 11. Gestión de PQR con sus cuatro estados.

9.4 Panel del empleado

El empleado ve una versión recortada: seis indicadores en vez de doce, y sin acceso a la gestión de usuarios ni a los ingresos globales. La restricción no es solo visual: si intentara llamar directamente al endpoint de usuarios, el backend respondería 403.

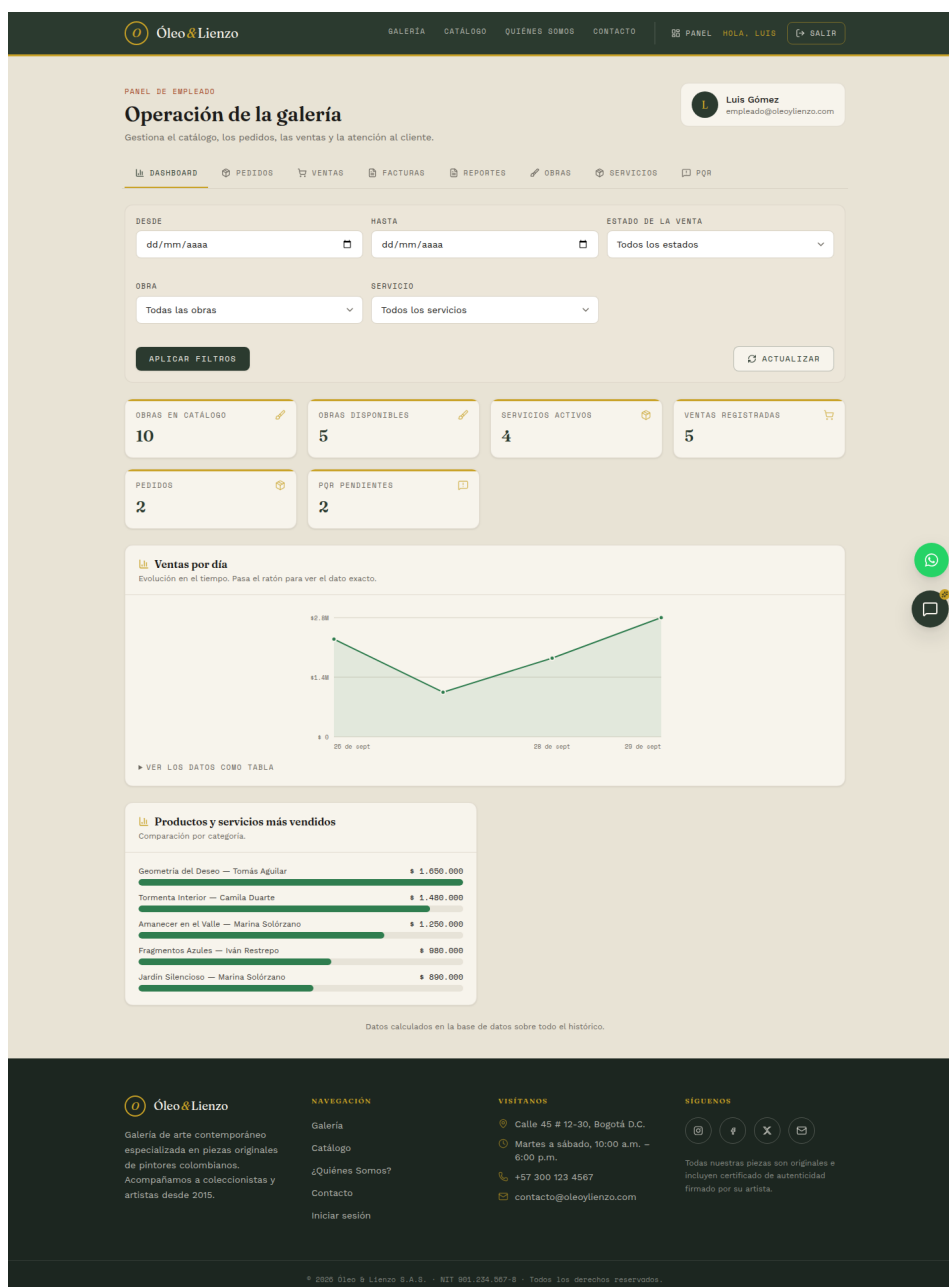


Figura 12. Panel del empleado, con alcance reducido.

9.5 Panel del cliente

El cliente ve únicamente su propia actividad: sus pedidos, sus compras, lo que lleva invertido, sus facturas y sus PQR abiertas.

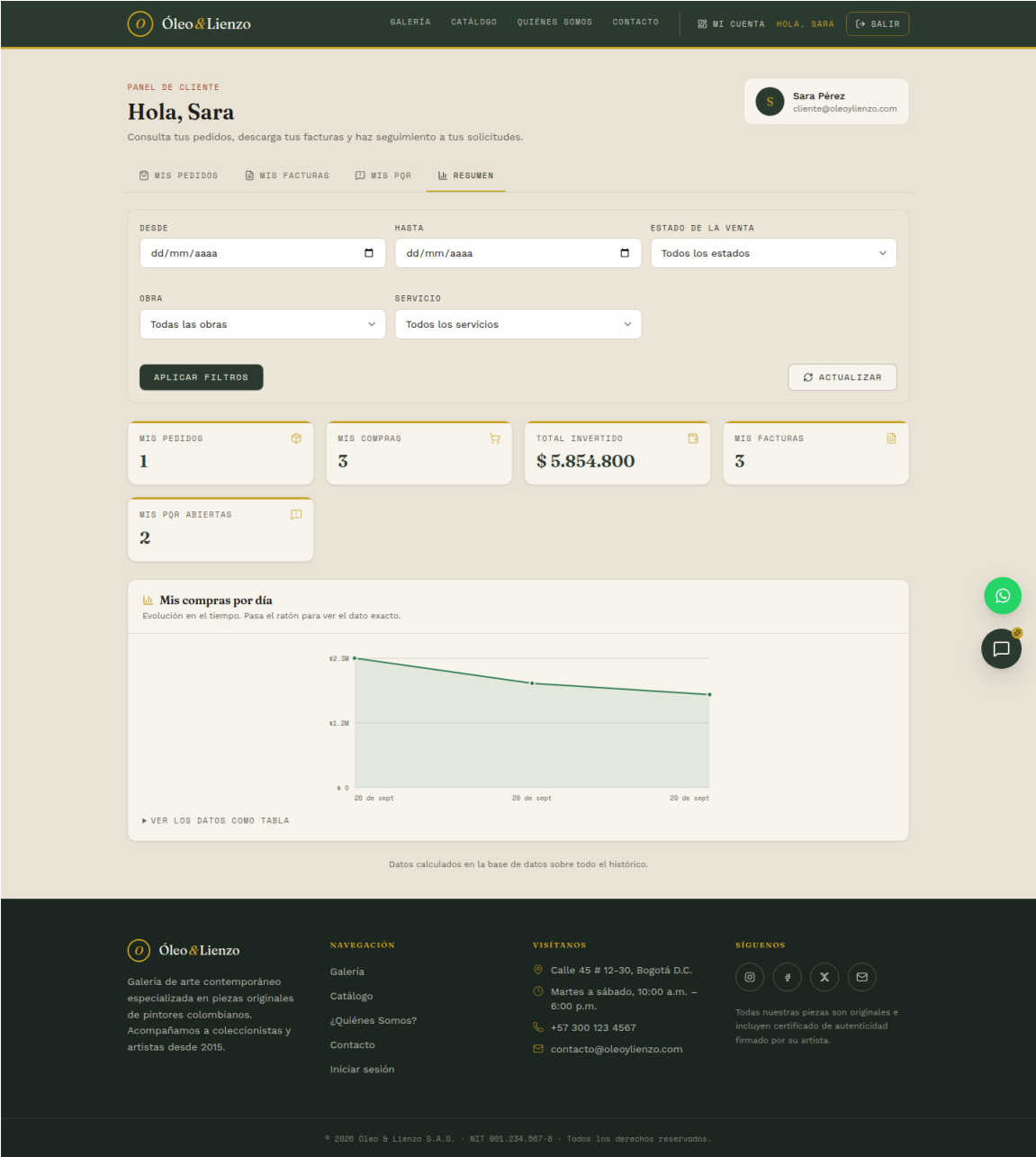


Figura 13. Panel del cliente con su actividad.

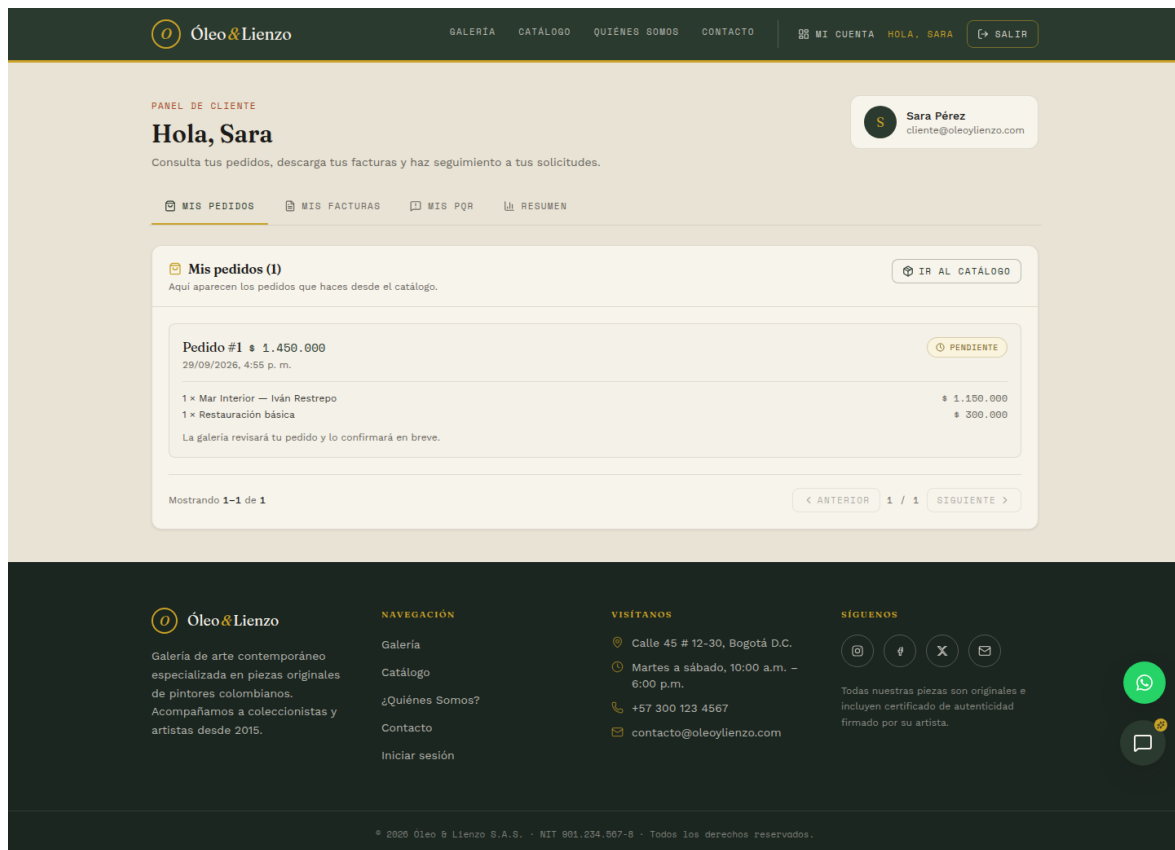


Figura 14. Mis pedidos, con el detalle de cada uno.

9.6 Chatbot de atención

El botón flotante de la esquina inferior derecha abre el chatbot, disponible también para visitantes sin cuenta. Responde usando el catálogo real como contexto, orienta sobre el proceso de compra y recibe solicitudes de PQR. Cuando el proveedor de IA no está disponible, responde en modo local y **lo declara abiertamente** en vez de hacer pasar la respuesta de respaldo por una respuesta de la IA.

9.7 Documentación interactiva de la API

En /docs puede probarse cada operación desde el navegador. El botón **Authorize** permite iniciar sesión y, a partir de ahí, las rutas protegidas envían el token automáticamente.

Óleo & Lienzo API

OpenAPI

API REST de Óleo & Lienzo, una galería de arte que vende pinturas originales y servicios asociados.

Arquitectura

React + Vite - FastAPI - PostgreSQL

Cómo autenticarse en esta documentación

1. Pulsa Authorize (arriba a la derecha).

2. Introduce el correo en el campo `username` y la contraseña.

3. Las rutas protegidas ya envían `Authorization: Bearer <token>`.

Usuarios de prueba que crea `python seed.py`:

Rol	Correo	Contraseña
Administrador	<code>admin@oleo lienzo.com</code>	<code>Admin1234</code>
Empleado	<code>empleado@oleo lienzo.com</code>	<code>Empleado123</code>
Cliente	<code>cliente@oleo lienzo.com</code>	<code>Cliente123</code>

Roles

Rol	Alcance
Administrador	Todo: usuarios, catálogo, ventas, facturas, PQR y diagnóstico.
Empleado	Catálogo, pedidos, ventas, facturas y PQR. Sin gestión de usuarios.
Cliente	Su propio catálogo, pedidos, compras, facturas y PQR.

La autorización definitiva siempre la decide el backend.

Códigos de respuesta

Código	Significado
200 / 201 / 204	Operación correcta
401	Falta el token o no es válido
403	Autenticado pero sin permisos
404	El recurso no existe
409	Conflicto (duplicado o resticción única)
422	Validación o regla de negocio incumplida
429	Demasiadas peticiones

Todos los errores comparten el mismo formato: `{ "error": "...", "detail": "..." }`.

Configuración

Los datos sensibles se leen de variables de entorno (ver `.env.example`). El IVA aplicado es del 19%.

Contact Óleo & Lienzo S.A.S.

Uso académico — Ficha 3406211, SENA

Authorize

Filter by tag

Autenticación

Registro, inicio de sesión con JWT y gestión de la contraseña.

Usuarios

CRUD de usuarios, roles y permisos. Solo administrador.

Obras / Productos

Catálogo de obras. Lectura pública, escritura restringida.

Servicios

Enmarcado, restauración y envío asegurado.

Pedidos

Órdenes del cliente. Confirmar un pedido genera su venta.

Ventas

Registro e historial comercial con filtros y desglose de IVA.

Facturación

Emisión, consulta y descarga de facturas en PDF.

Reportes

Reporte diario de ventas en JSON, PDF y Excel.

Dashboard

Indicadores y gráficos calculados en la base de datos, por rol.

PQR

Peticiones, quejas, reclamos y sugerencias.

Chatbot IA

Asistente de atención al cliente con Grog.

Pagos (Stripe)

Checkout alzado y webhook firmado.

Inteligencia Artificial

Modelo propio de precios y redacción asistida.

Sistema

Salud y diagnóstico real de cada dependencia.

Schemas

Figura 15. Documentación automática generada por FastAPI.

10. Pruebas realizadas

10.1 Estrategia

El proyecto cuenta con **164 pruebas automatizadas** escritas con pytest, que se ejecutan contra la API completa mediante un cliente HTTP asíncrono. No prueban funciones sueltas: recorren el mismo camino que recorrería el navegador, desde la petición hasta la base de datos.

Dos decisiones hacen que la suite sea fiable. La primera: **cada prueba recibe una base de datos recién creada**, de modo que ninguna puede contaminar a la siguiente. La segunda: **las mismas pruebas se ejecutan sobre SQLite y sobre PostgreSQL**; SQLite por velocidad en el día a día, PostgreSQL porque es el motor real y hay diferencias de dialecto que solo aparecen allí.

10.2 Qué cubre cada archivo

Archivo	Qué verifica
test_auth.py	Registro con validaciones, correo y documento duplicados (409), inicio de sesión, token caducado, firma manipulada y recuperación de contraseña.
test_usuarios.py	CRUD completo, cambio de estado, que el administrador no pueda desactivarse a sí mismo y que un cliente no acceda a la gestión de usuarios.
test_catalogo.py	CRUD de obras y servicios, diferencia real entre PUT y PATCH, filtros, paginación y 404 con mensaje claro.
test_comercial.py	Pedidos, confirmación que genera la venta, cálculo del IVA, control de inventario, máquinas de estado y emisión de facturas.
test_sistema_reportes.py	Reporte diario en JSON, PDF y Excel; dashboards por rol; los seis filtros; y que filtrar por obra no duplique los importes.
test_pqr_chatbot.py	Radicación con radicado, transiciones de estado, respuesta del personal y persistencia de las conversaciones del chatbot.
test_seguridad.py	Cabeceras defensivas, limitación de intentos de inicio de sesión y formato uniforme de los errores.
test_dobles_externos.py	Groq y el modelo de IA sustituidos por dobles: respuesta correcta, tiempo de espera agotado, HTTP 500, respuesta malformada, degradación declarada y que la clave nunca viaje al cliente.

10.3 Casos de prueba destacados

Tres pruebas merecen mención porque protegen errores que serían difíciles de detectar a ojo:

- **El filtro por obra no duplica los importes.** Si el filtro se resolviera con un JOIN al detalle, una venta de dos líneas aparecería dos veces y tanto el conteo como los totales se duplicarían. La prueba crea justamente esa venta y comprueba que el importe coincide exactamente.
- **Un cliente no puede espiar el dashboard de otro.** El endpoint acepta un parámetro `cliente_id`, pero para el rol cliente se ignora y se fuerza el propio. La prueba intenta el acceso cruzado y verifica que los datos devueltos son los suyos.
- **El chatbot no finge.** Cuando el proveedor de IA falla, el servicio responde con el modo local, y la prueba comprueba que `generado_por_ia` vale `false`: la aplicación no hace pasar una respuesta de respaldo por una respuesta de la inteligencia artificial.

10.4 Cómo ejecutarlas y qué resultado dan

```
cd backend
source venv/bin/activate

pytest -q # sobre SQLite
```

```
# 164 passed

TEST_DATABASE_URL=postgresql+psycopg://oleo:oleo@localhost:5432/oleo_test \
pytest -q # sobre PostgreSQL
# 164 passed
```

Advertencia importante: la variable TEST_DATABASE_URL debe apuntar a una base de datos *distinta* de la de desarrollo. La suite borra y recrea las tablas en cada prueba, de modo que apuntarla a la base de trabajo destruiría sus datos.

10.5 Pruebas manuales complementarias

Tipo	Herramienta	Alcance
Pruebas de la API	Postman	Colección de 60 peticiones, una por operación, con los cinco verbos.
Recorrido de interfaz	Playwright + Chrome	Inicio de sesión con los tres roles y recorrido de todas las secciones.
Inspección de la base	psql	Comprobación de tablas, restricciones y de que las contraseñas son hashes bcrypt.
Estilo del código	ruff	Comprobación de estilo y formato en 86 archivos de Python.

11. Conclusiones y recomendaciones

11.1 Aprendizajes del proceso

- **La validación no se delega al frontend.** Es cómodo pensar que si el formulario ya validó, el backend puede confiar. No es así: quien llame a la API directamente se salta el formulario entero. Cada dato se valida dos veces, y la que cuenta es la del servidor.
- **El dinero no se calcula con números de coma flotante.** Empezar con `float` y ver cómo las facturas se descuadraban por céntimos fue la lección más concreta del proyecto. Todo el cálculo monetario pasó a `Decimal` con redondeo explícito.
- **Separar las capas se paga solo cuando algo cambia.** Mantener la lógica de negocio fuera de las funciones de ruta parecía burocracia hasta que hubo que reutilizarla desde el seed y desde las tareas en segundo plano.
- **Una integración externa debe poder fallar.** El chatbot no se diseñó suponiendo que el proveedor responde siempre: se diseñó decidiendo qué hacer cuando no responde. Degradar y decirlo es mejor que quedarse en blanco, y mucho mejor que fingir.
- **Las pruebas no son un trámite final.** Las que más valor dieron fueron las que se escribieron al detectar un comportamiento dudoso, no las añadidas al final para subir la cobertura.

11.2 Limitaciones conocidas

Reconocerlas es parte del trabajo técnico; ocultarlas solo retrasa el momento en que aparecen.

- El chatbot depende de un proveedor externo con plan gratuito, sujeto a límites de uso. Cuando se agotan, responde el modo local.
- La generación de PDF y Excel es síncrona. Con reportes de miles de ventas convendría moverla a una tarea en segundo plano con descarga diferida.
- No hay migraciones de esquema (Alembic). Un cambio en los modelos exige recrear las tablas, lo cual es aceptable en formación pero no en producción.
- El modelo de sugerencia de precios es una regresión lineal con dos variables entrenada con diez obras. Ilustra la integración de un modelo propio; su precisión no es el objetivo.

11.3 Recomendaciones para una versión futura

Prioridad	Recomendación	Por qué
Alta	Incorporar Alembic para versionar el esquema.	Permite evolucionar la base en producción sin perder datos.
Alta	Rotar las credenciales de prueba antes de cualquier uso real.	Las que crea el seed son públicas: están en este documento.
Media	Mover la generación de reportes a una cola de tareas.	Evita bloquear la petición cuando el volumen crezca.
Media	Añadir paginación por cursor en el historial de ventas.	El desplazamiento por OFFSET se degrada con muchas filas.
Media	Registrar métricas y trazas del tiempo de respuesta.	Ya existe el X-Request-ID; falta explotarlo.
Baja	Cachear el catálogo, que cambia poco y se consulta mucho.	Reduciría la carga de la base en la zona pública.

11.4 Conclusión

El proyecto cumple el objetivo planteado: una aplicación web full stack, funcional y verificada, que cubre el ciclo comercial completo de una galería de arte —del catálogo a la factura— con seguridad basada en roles, analítica alimentada por la base de datos e inteligencia artificial integrada en dos

frentes.

Más allá de las funcionalidades, el resultado que mejor resume el trabajo es que **164 pruebas automatizadas pasan sobre dos motores de base de datos distintos**, y que cada afirmación de este manual puede comprobarse ejecutando el código. La documentación no describe lo que el proyecto pretendía ser: describe lo que hace, y deja a mano la forma de verificarlo.

12. Anexos

12.1 Repositorio y estructura de entrega

Recurso	Ubicación
Repositorio	https://github.com/vegajerson88-droid/Oleo-Lienzo
Backend	backend/
Frontend	frontend/
Script SQL	backend/sql/schema_postgresql.sql
Colección de Postman	postman/Oleo-Lienzo.postman_collection.json
Pruebas	backend/tests/
Capturas de este manual	docs/manual/capturas/

12.2 Documentación complementaria en el repositorio

Documento	Contenido
README.md	Instalación, ejecución, variables de entorno y estructura.
docs/disenio-api.md	Tabla de diseño recurso–verbo–ruta–código de las 60 operaciones.
docs/matriz-cumplimiento.md	Verificación requisito por requisito de los cinco entregables.
docs/comparativa-fastapi-drf.md	Análisis comparativo entre FastAPI y Django REST Framework.
docs/despliegue.md	Guía de despliegue en la nube paso a paso.
docs/evidencias.md	Guion de sustentación y evidencias de funcionamiento.

12.3 Scripts de apoyo

Varios artefactos del proyecto se generan en vez de escribirse a mano, precisamente para que no puedan quedar desfasados respecto al código:

Script	Qué genera
seed.py	Datos de ejemplo y entrenamiento del modelo de IA.
scripts/generar_sql.py	El script SQL, a partir de los modelos SQLAlchemy.
scripts/generar_postman.py	La colección de Postman, desde el esquema OpenAPI.
scripts/generar_diseno_api.py	La tabla de diseño de la API.
scripts/capturar_evidencias.py	Las capturas de pantalla de este manual.
scripts/generar_manual_tecnico.py	Este mismo documento.

12.4 Glosario

Término	Significado en este proyecto
JWT	Token firmado que identifica al usuario en cada petición. Lleva su correo, su rol y su fecha de caducidad.
bcrypt	Algoritmo de hashing de contraseñas con sal propia. Del hash almacenado no puede recuperarse la contraseña.
ORM	Capa que traduce entre las tablas de la base y los objetos de Python. Aquí, SQLAlchemy.

Término	Significado en este proyecto
Pydantic	Biblioteca que valida los datos que entran y salen de la API a partir de anotaciones de tipo.
CORS	Mecanismo del navegador que decide qué orígenes pueden llamar a la API. Aquí es una lista explícita.
PQR	Peticiones, quejas y reclamos: el canal formal de atención al cliente.
Idempotente	Que puede ejecutarse varias veces con el mismo resultado. El seed lo es.
Degradar	Seguir funcionando con capacidad reducida cuando un servicio externo falla, en vez de romperse.

Jerson Jesús Vega Naranjo Aprendiz · Ficha 3406211	César Augusto Moreno Mena Instructor técnico